ECE131a Extra Credit Project
Distributed Detection in Gaussian Noise

Wesley Hon

June 5, 2026

Contents

1 Introduction

1.1 Background

This problem is adapted from the results in [1] and [2]. A scalar signal $S \in \{-m, m\}$ is transmitted from a satellite to a field of n sensors on the surface of the earth. S communicates an important message, and is equally likely to take either value so that

$$P_S(s) = \begin{cases} \frac{1}{2}, & \text{for } s = -m, \\ \frac{1}{2}, & \text{for } s = m, \\ 0, & \text{otherwise.} \end{cases}$$

Sensor i observes random variable

$$X_i = S + Z_i$$

where the random variables Z_i are independent and identically distributed according to a standard normal distribution $\mathcal{N}(0, 1)$. This project explores how to efficiently and reliably estimate S from the vector of observations

$$\mathbf{X} = [X_1, X_2, \ldots, X_n].$$

Any estimate $\hat{S}$ will naturally depend on the observables. A *statistic* is a function of the set of observables that we use in preparing our estimate. For this problem we will be especially interested in the statistic

$$T_n = \frac{1}{n} \sum_{i=1}^{n} X_i,$$

which is sometimes called the sample mean.
A *sufficient statistic* of a set of observations for estimation of a particular random variable is one that allows as good of an estimate of the random variable as the original observations themselves. T_n is a sufficient statistic of $\mathbf{X}$ for S if and only if

$$f_{X|T,S}(x|t, s) = f_{X|T}(x|t).$$

In our specific case, this turns out to be true so that the sample mean is a sufficient statistic of $\mathbf{X}$ for S. We can use the sample mean T_n to estimate S and be confident that we have not lost any information. Specifically, we will estimate S as follows:

$$\hat{S} = \begin{cases} m, & \text{if } T_n \geq 0, \\ -m, & \text{otherwise.} \end{cases}$$

1.2 Report Organization

The project is 25 project points which is equivalent to 5% of the total grade percentage as extra credit. The project must be completed to earn an A+ grade.
There are 8 problems, each answer must have a Matlab plot to back up the answer.

Project Statement:
ECE 131a Spring 2026 Extra Credit Project, Distributed Detection in Gaussian Noise

2 Problem 1

3pts.

Suppose that $m = 0.5$. Use Matlab function `semilogy` to plot the probability of error $P(\hat{S} \neq S)$ as a function of the number of sensors n for $1 \leq n \leq 100$. The Matlab function `qfunc` is your friend for this problem. How many sensors do you need so that $P(\hat{S} \neq S) < 10^{-3}$? Call this number n_g (as in the number of Gaussian samples).

For this problem, each sensor observes

$$X_i = S + Z_i$$

where $S \in \{-m, m\}$ and the noise variables Z_i are independent and identically distributed according to a standard normal distribution $N(0, 1)$. The signal amplitude is given as $m = 0.5$.

The detector uses the sample mean

$$T_n = \frac{1}{n} \sum_{i=1}^{n} X_i$$

and makes the decision

$$\hat{S} = \begin{cases} m, & T_n \geq 0 \\ -m, & T_n < 0 \end{cases}$$

The goal is to determine the probability of error as a function of the number of sensors n and find the minimum number of sensors required so that the probability of error is less than 10^{-3}.

2.1 Distribution of the Sample Mean

Starting with the definition of the sample mean,

$$T_n = \frac{1}{n} \sum_{i=1}^{n} (S + Z_i)$$

Since S is constant across all sensors,

$$T_n = S + \frac{1}{n} \sum_{i=1}^{n} Z_i$$

Because the noise variables Z_i are independent Gaussian random variables with mean 0 and variance 1, the average noise term has distribution

$$\frac{1}{n} \sum_{i=1}^{n} Z_i \sim N\left(0, \frac{1}{n}\right)$$

Therefore, conditioned on the transmitted signal S,

$$T_n|S = m \sim N\left(m, \frac{1}{n}\right)$$

and

$$T_n|S = -m \sim N\left(-m, \frac{1}{n}\right)$$

Thus, the sample mean remains to be Gaussian with mean equal to the transmitted signal (m) and variance equal to $1/n$.

2.2 Probability of Error

An error occurs when the detector chooses the wrong hypothesis.
Considering the case $S = m$, the detector makes an error whenever $T_n < 0$. Therefore,

$$P(\text{error}|S = m) = P(T_n < 0|S = m)$$

Since $T_n|S = m$ is Gaussian with mean m and variance $1/n$, we are able to standardize the random variable:

$$P(\text{error}|S = m) = P\left(\frac{T_n - m}{1/\sqrt{n}} < \frac{0 - m}{1/\sqrt{n}}\right)$$

$$= P(Z < -m\sqrt{n})$$

where $Z \sim N(0, 1)$.
Using the Gaussian Q-function,

$$P(\text{error}|S = m) = Q(m\sqrt{n})$$

By symmetry,

$$P(\text{error}|S = -m) = Q(m\sqrt{n})$$

Since both of the hypotheses are equally likely,

$$P(\text{error}) = \frac{1}{2}P(\text{error}|S = m) + \frac{1}{2}P(\text{error}|S = -m)$$

$$= Q(m\sqrt{n})$$

Substituting $m = 0.5$:

$$P(\text{error}) = Q(0.5\sqrt{n})$$

This expression above is the theoretical probability of error for any number of sensors n.

2.3 MATLAB

Listing 1: Matlab Code: Problem 1

```
clc;
clear;
close all;

%% Problem 1
% Distributed Detection in Gaussian Noise
% P(error) = Q(m*sqrt(n))

m = 0.5;                % Signal amplitude
n = 1:100;              % Number of sensors

% Compute probability of error, using qfunc
Pe = qfunc(m*sqrt(n));

% Error threshold (10^-3)
threshold = 1e-3;

% Find smallest n satisfying Pe < 10^-3
ng = find(Pe < threshold, 1, 'first');

%% Create Figure
figure;

semilogy(n, Pe, 'LineWidth', 2); % using semilogy
hold on;
grid on;

% Threshold line
yline(threshold,'--','10^{-3}','LineWidth',1.5);

% Mark ng
semilogy(ng, Pe(ng), 'o', ...
    'MarkerSize', 8, ...
    'LineWidth', 2);

xlabel('Number of Sensors, n');
ylabel('Probability of Error');
title('Probability of Error vs. Number of Sensors (m = 0.5)');

legend('P_e = Q(0.5\sqrt{n})', ...
       'Threshold', ...
       sprintf('n_g = %d',ng), ...
       'Location','southwest');

%% Displaying Results (print)
fprintf('\n');
fprintf('-----------------------------------\n');
fprintf('Problem 1 Results\n');
```

```
fprintf('-----------------------------------\n');
fprintf('Signal amplitude m = %.1f\n',m);
fprintf('Required error probability < 10^-3\n');
fprintf('Minimum number of sensors n_g = %d\n',ng);
fprintf('Probability of error at n_g = %.6e\n',Pe(ng));

if ng > 1
    fprintf('Probability of error at n = %d is %.6e\n', ...
        ng-1, Pe(ng-1));
end
fprintf('-----------------------------------\n');
```

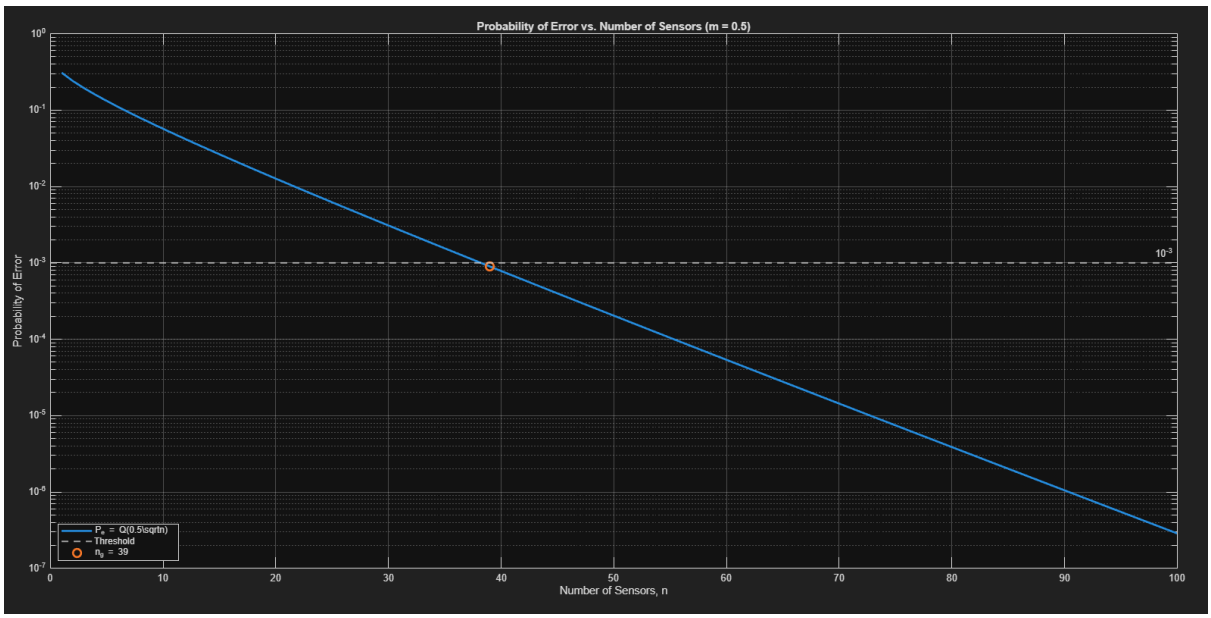

Figure 1: Probability of error as a function of the number of sensors for $m = 0.5$.

```
-----------------------------------
Problem 1 Results
-----------------------------------
Signal amplitude m = 0.5
Required error probability < 10^-3
Minimum number of sensors n_g = 39
Probability of error at n_g = 8.966136e-04
Probability of error at n = 38 is 1.027359e-03
-----------------------------------
```

Figure 2: MATLAB command window output for Problem 1.

The theoretical probability of error derived in the previous section is

$$P(\hat{S} \neq S) = Q(0.5\sqrt{n}).$$

This expression was evaluated in MATLAB for values of n (the number of sensors) ranging from 1 to 100. The built-in `qfunc` function was used to compute the Gaussian Q-function, and the results were plotted using the `semilogy` command. A logarithmic scale was used for the vertical axis because the probability of error decreases rapidly as the number of sensors increases.

Figure 1 shows the probability of error as a function of the number of sensors. The graph shows that the probability of error decreases steadily as n increases in the logarithmic scale. This makes sense because averaging more independent sensor measurements reduces the variance of the sample mean, making it easier to distinguish between the two possible transmitted signal values, thereby ensuring our confidence in an accurate signal.

A horizontal reference line at 10^{-3} was added to the plot to show the required error threshold. From the graph, the error probability crosses this threshold between $n = 38$ and $n = 39$. MATLAB was then used to find the smallest value of n satisfying

$$P(\hat{S} \neq S) < 10^{-3}.$$

The numerical results show that

$$P_e(38) = 1.027359 \times 10^{-3}$$

and

$$P_e(39) = 8.966136 \times 10^{-4}.$$

Therefore, $n = 38$ is not enough, since its error probability is still above 10^{-3}. However, $n = 39$ is sufficient because its error probability is below 10^{-3}. Thus, the minimum number of sensors required is

$$\boxed{n_g = 39}$$

3 Problem 2

4pts.

Simulate this sensor system with the n_g sensors as follows: For each simulation run, generate n_g observations X_i using $S = 0.5$ and `normrnd` to generate Z_i and use them to compute the sufficient statistic T_{n_g}. Then use T_{n_g} to compute $\hat{S}$ and note whether the estimate is correct or is an error. Perform simulation runs until you have observed 200 errors. Keep track of the total number K of simulation runs. Report your estimated error rate as $200/K$. If everything is working well, your simulation point should match the probability of error you calculated in the previous part for n_g observations, i.e., a number close to 10^{-3}. Your simulation may take a few minutes to complete since you will need approximately 200,000 runs.

3.1 Preliminary Remarks

In this problem, we should verify the result calculated in Problem 1 using a Matlab simulation. It was figured out that the minimum number of sensors required to make the probability of error less than 10^{-3} was found to be

$$n_g = 39.$$

The transmitted signal is fixed as

$$S = 0.5.$$

Each sensor observes

$$X_i = S + Z_i,$$

where

$$Z_i \sim N(0, 1).$$

For each simulation run, $n_g = 39$ independent noise samples are generated, producing 39 sensor observations. The sample mean is then computed as

$$T_{n_g} = \frac{1}{n_g} \sum_{i=1}^{n_g} X_i.$$

The detector uses the rule

$$\hat{S} = \begin{cases} 0.5, & T_{n_g} \geq 0, \\ -0.5, & T_{n_g} < 0. \end{cases}$$

Since the true value is $S = 0.5$, an error occurs whenever the detector decides $\hat{S} = -0.5$. There is a deviation. This happens when the sample mean is negative:

$$T_{n_g} < 0.$$

The simulation is repeated until 200 errors are observed. If K is the total number of simulation runs required to observe those 200 errors, then the simulated probability of error is estimated as

$$\hat{P}_e = \frac{200}{K}.$$

This estimate should be close to the theoretical value $P_e(39)$.

$$P_e(39) = Q(0.5\sqrt{39}) \approx 8.966136 \times 10^{-4}.$$

Because the error probability is close to 10^{-3}, approximately

$$K \approx \frac{200}{8.966136 \times 10^{-4}}$$

simulation runs are expected. This gives

$$\boxed{K \approx 223{,}061}$$

Therefore, the simulation should require around $220,000$ runs before reaching 200 errors. The exact value of K may vary each time the simulation is run because the Gaussian noise samples are random. However, if the simulation is correct, the estimated probability of error $200/K$ should be close to 10^{-3}, confirming the theoretical result that I calculated in Problem 1.

3.2 Matlab Code

Listing 2: Matlab Code: Problem 1

```
clc;
clear;
close all;

%% Problem 2
% Simulation of full-precision sensor system

m = 0.5;                    % Signal amplitude
S = 0.5;                    % Transmitted signal for simulation
ng = 39;                    % Number of sensors calculated from p1

target_errors = 200;        % Stop simulation after 200 errors
num_errors = 0;             % Error counter
K = 0;                      % Total simulation runs

%% Simulation Loop

while num_errors < target_errors

    % Increase simulation run counter
    K = K + 1;

    % Generate ng independent Gaussian noise samples
```

```
    Z = normrnd(0, 1, [1, ng]);

    % Generate sensor observations
    X = S + Z;

    % Compute sufficient statistic
    T = mean(X);

    % Decision rule
    if T >= 0
        S_hat = 0.5;
    else
        S_hat = -0.5;
    end

    % Check if an error occurred
    if S_hat ~= S
        num_errors = num_errors + 1;
    end
end

%% Compute Error Probability

Pe_sim = target_errors / K;
Pe_theory = qfunc(m * sqrt(ng));

%% Display Results

fprintf('\n');
fprintf('-----------------------------------\n');
fprintf('Problem 2 Simulation Results\n');
fprintf('-----------------------------------\n');
fprintf('Number of sensors n_g = %d\n', ng);
fprintf('Target number of errors = %d\n', target_errors);
fprintf('Total number of simulation runs K = %d\n', K);
fprintf('Estimated error probability = 200/K = %.6e\n', Pe_sim);
fprintf('Theoretical error probability = %.6e\n', Pe_theory);
fprintf('-----------------------------------\n');
```

```
-----------------------------------
Problem 2 Simulation Results
-----------------------------------
Number of sensors n_g = 39
Target number of errors = 200
Total number of simulation runs K = 220803
Estimated error probability = 200/K = 9.057848e-04
Theoretical error probability = 8.966136e-04
-----------------------------------
```

Figure 3: Problem 2 MATLAB simulation output (Run 1).

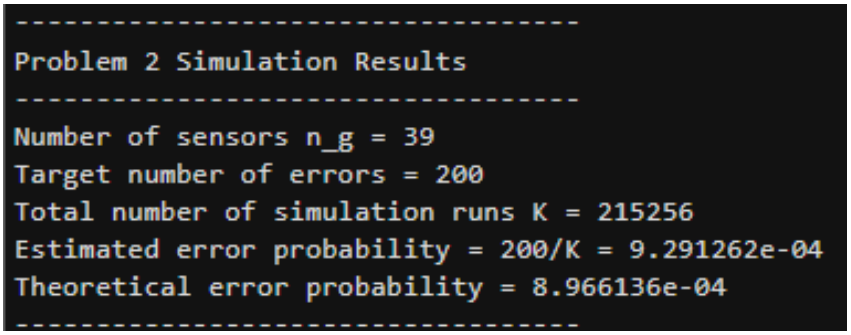

Figure 4: Problem 2 MATLAB simulation output (Run 2).

3.3 Interpretation of Results

The MATLAB simulation was performed twice using the full-precision sensor system with $n_g = 39$ sensors. In each trial, the simulation continued until 200 detection errors were observed. The probability of error was then estimated using

$$\hat{P}_e = \frac{200}{K},$$

where K is the total number of simulation runs required to accumulate 200 errors.

For Run 1, the simulation required

$$K = 220{,}803$$

trials, resulting in an estimated probability of error of

$$\hat{P}_{e,1} = 9.057848 \times 10^{-4}.$$

For Run 2, the simulation required

$$K = 215{,}256$$

trials, resulting in an estimated probability of error of

$$\hat{P}_{e,2} = 9.291262 \times 10^{-4}.$$

These values are very close to the theoretical probability of error obtained earlier,

$$P_e(39) = 8.966136 \times 10^{-4}.$$

The small differences between the simulated and theoretical values are expected because the simulation relies on randomly generated Gaussian noise samples. Each simulation run produces a different outcome of the noise process, causing slight variations in the estimated error probability. Despite these random fluctuations, both simulation results are within a few percent of the theoretical prediction.

The results therefore confirm the analysis from Problem 1. Both simulation trials produced error probabilities on the order of 10^{-3}, demonstrating that 39 sensors are sufficient to achieve the desired level of reliability. The close agreement between the theoretical and simulated probabilities of error verifies that the sample mean detector does accurately model the performance of the full-precision sensor system.

4 Problem 3

3pts.

We will refer to the sensor studied in problems 1–2 as a "full precision" sensor. Now, let's suppose that it is resource intensive to sense the real numbers X_i with full precision. We want to deploy less expensive sensors that merely detect whether $X_i \geq 0$. These sensors each collect

$$B_i = \begin{cases} 1 & \text{if } X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

We are fortunate that $U_n = \sum_{i=1}^{n} B_i$ is a sufficient statistic of B for S, so that we can base our estimate $\hat{S}$ on this sum.

What kind of random variable is B_i? What kind of random variable is U_n? For our case of interest, where $S = \pm 0.5$ and $X_i = S + Z_i$ with the random variables Z_i independent and identically distributed according to a standard normal distribution $\mathcal{N}(0, 1)$, find the value of $P(B_i = 1 \mid S = 0.5)$, which we will call p.

We need a decision rule to decide our estimate $\hat{S}$ as a function of U_n. We will use the "maximum likelihood" principle to choose our rule. Specifically, choose the value of $\hat{S}$ the rule that maximizes the probability of the observed value of U_n given $S = \hat{S}$.

$$\hat{S} = \begin{cases} 0.5 & \text{if } P_{U|S}(u|0.5) > P_{U|S}(u|-0.5) \\ -0.5 & \text{otherwise.} \end{cases}$$

Show that the maximum-likelihood decoder turns out to be a majority vote decoder. It turns out that how you handle ties does not affect the total probability of error, but it could favor one of the outcomes.

---

5 Problem 4

3pts.

Use the Matlab function `semilogy` to plot the probability of error $P(\hat{S} \neq S)$ as a function of the number of sensors n for $1 \leq n \leq 100$ for the system based on sensors that provide B_i. Add this to your plot from problem 2 so that you have a single plot with two curves. Label your axes. Include a legend. Take pride in your data. For the binary sensor, you only need to plot probability of error for odd values of n, so you don't need to worry about breaking ties. The Matlab function `binocdf` is your friend for this problem. How many sensors, considering only odd numbers, do you need so that $P(\hat{S} \neq S) < 10^{-3}$? Call this number n_b (as in the number of Bernoulli samples in the binomial random variable.)

---

6 Problem 5

3pts.

Simulate this binary sensor system with n_b sensors as follows: For each simulation run, generate n_b observations X_i as before and then compute the associated B_i values and use them to compute the sufficient statistic U_{n_b}. Then use U_{n_b} to compute $\hat{S}$ and note whether the estimate is correct or is an error. Perform simulation runs until you observe 200 errors. Keep track of the total number K of simulation runs. Report your estimated error rate as $\frac{200}{K}$. If everything is working well, your simulation point should match the probability of error you calculated in the previous part for n_b observations, i.e. a number close to 10^{-3}.

---

7 Problem 6

3pts.

If the binary sensor that we studied in problems 3–5 costs half as much as the full-precision sensor we studied in problems 1–2, which system costs less to achieve probability of error 10^{-3}? Based on the slopes of the lines in your plot, would the same system also be the most cost effective for lower probabilities of error, assuming binary sensors costs half as much as the full-precision sensors?

---

8 Problem 7

3pts.

As it turns out, the binary sensors as we have defined them above have a peculiar property that for an odd number n, the probability of error for n sensors is the same as the probability of error for $n+1$ sensors. Confirm this with a few calculations using `binocdf`. You can break ties, for example, by always calling them $\hat{s} = m$. Show mathematically that the probability of error for n sensors is the same as the probability of error for $n+1$ sensors for any odd number n and any value of p.

---

9 Problem 8

3pts.

Devise a modification to the sensor for the even-valued n case that will allow it to perform better than the case with one fewer sensor. If you can't think of a way, look at Fig. 7 in [1] and its discussion for a clue. For the value n_b that you identified in problem 4, optimize your modification to get the most improvement possible in the probability of error. It won't be too much better. You will never do better than $n_b + 2$ sensors using the standard approach we described above. Report the probability of error you obtain by applying your modification for the case of $n_b + 1$ sensors.

---

10 Conclusion